Object-Oriented Concepts with UML
Event handling

CCP6224

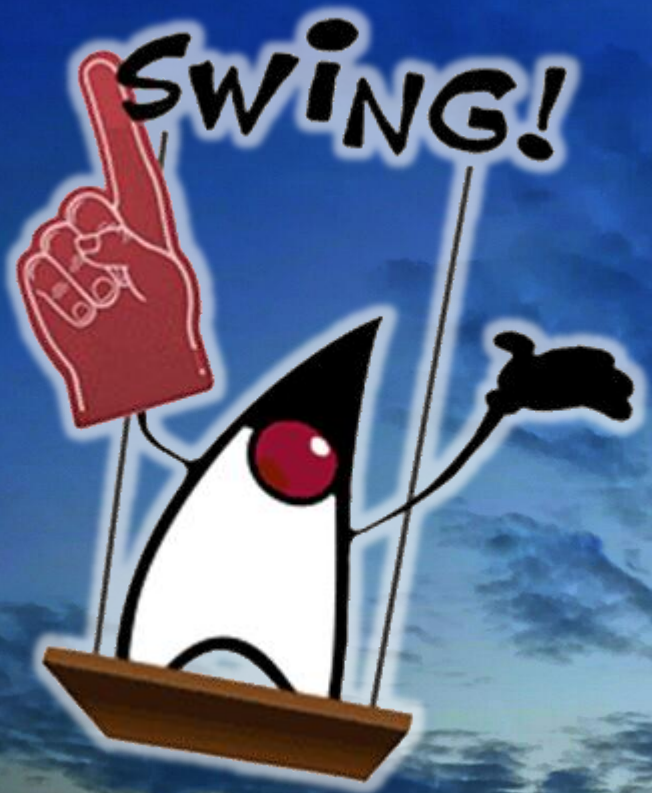
Object-oriented Analysis and Design

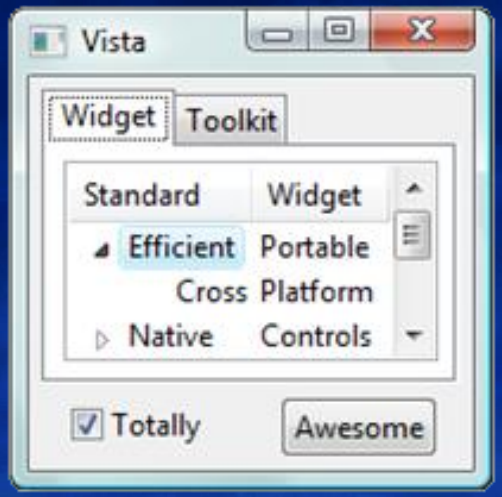
Lecture outcomes

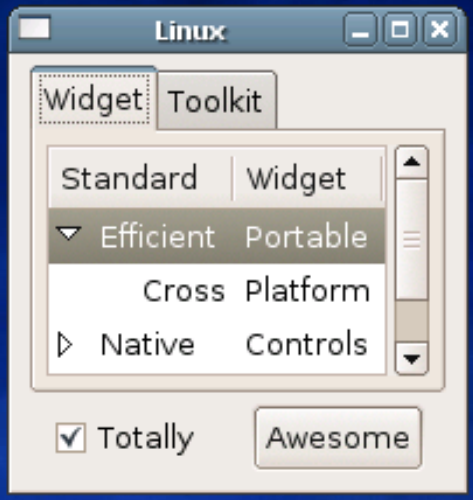
- At the end of this lecture, you should have learned about:
  - Components & Containers
    - Composite Design Pattern
  - Event handling
    - Listener Design Pattern

GUI Applications

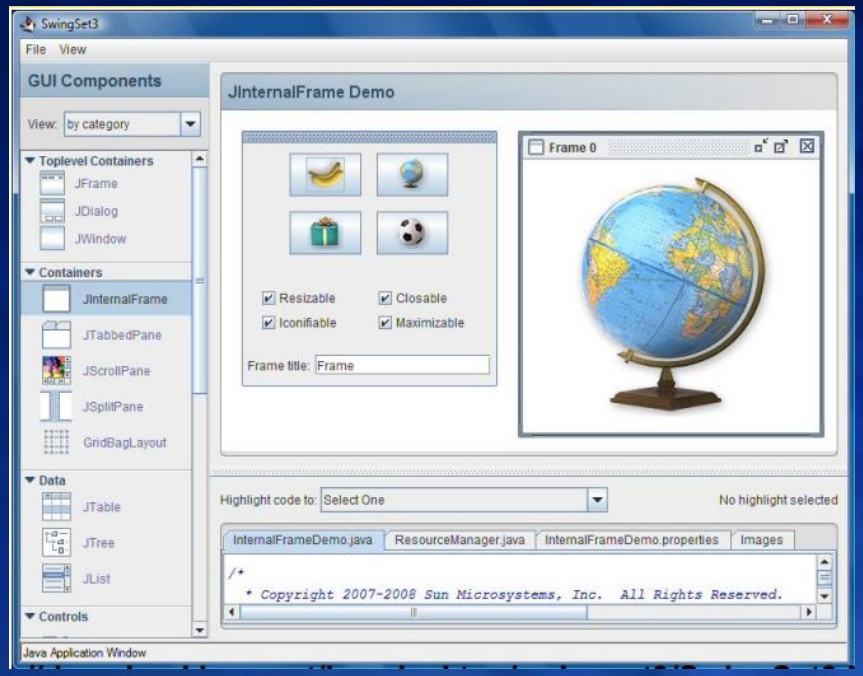
- Java graphical applications make use of the operating system capabilities to provide WIMP (windows, icons, menus and pointer) features
- Swing has been used for many years and is still very useful.
- JavaFX

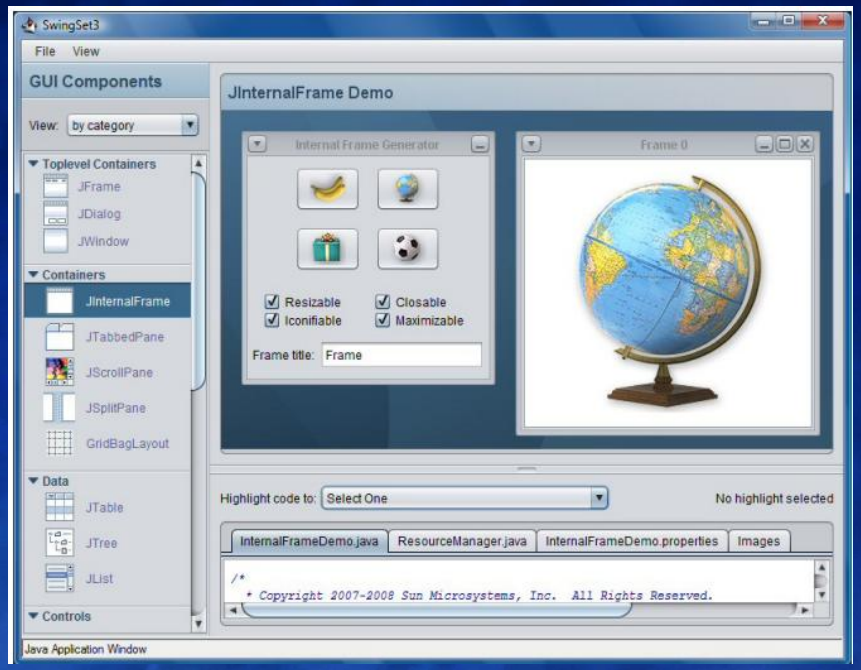


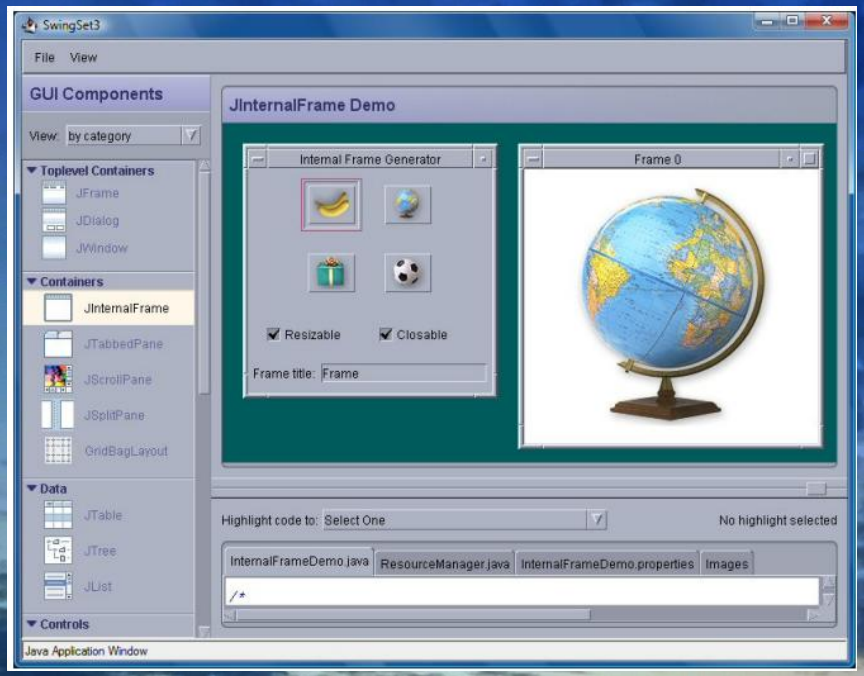




http://www.eclipse.org/swt/


SwingSet3
File View
GUI Components
JInternalFrame Demo
View: by category
Toplevel Containers
JFrame
JDialog
JWindow
Containers
JInternalFrame
JTabbedPane
JScrollPane
JSplitPane
GridBagLayout
Data
JTable
JTree
JList
Controls
Frame 0
Resizable
Closable
Iconifiable
Maximizable
Frame title: Frame
Highlight code to: Select One
No highlight selected
InternalFrameDemo.java
ResourceManager.java
InternalFrameDemo.properties
Images
/*
* Copyright 2007-2008 Sun Microsystems, Inc. All Rights Reserved.
Java Application Window


SwingSet3
File View
GUI Components
JInternalFrame Demo
View: by category
Toplevel Containers
JFrame
JDialog
JWindow
Containers
JInternalFrame
JTabbedPane
JScrollPane
JSplitPane
GridBagLayout
Data
JTable
JTree
JList
Controls
Internal Frame Generator
Frame 0
Resizable
Closable
Iconifiable
Maximizable
Frame title: Frame
Highlight code to: Select One
No highlight selected
InternalFrameDemo.java
ResourceManager.java
InternalFrameDemo.properties
Images
/*
* Copyright 2007-2008 Sun Microsystems, Inc. All Rights Reserved.
Java Application Window


SwingSet3
File View
GUI Components
JInternalFrame Demo
View: by category
Toplevel Containers
JFrame
JDialog
JWindow
Containers
JInternalFrame
JTabbedPane
JScrollPane
JSplitPane
GridBagLayout
Data
JTable
JTree
JList
Controls
Internal Frame Generator
Frame 0
Resizable
Closable
Frame title: Frame
Highlight code to: Select One
No highlight selected
InternalFrameDemo.java
ResourceManager.java
InternalFrameDemo.properties
Images
/*
Java Application Window

Starting with Java GUI

- Swing requires the `javax.swing.*` package but can work alongside `java.awt` and `java.event`
- Java GUI has two types of elements – *components* and *containers*
    - Graphical applications will use containers and components
    - Command line applications do not use any

Components and Containers

- Components
  - elementary GUI objects like button, text field etc.
  - components can only exist inside containers → use the **`add()`** method of a container

- Containers
  - holds components in a specific layout
  - can hold other containers/components
  - divided into 2: top-level and secondary-level containers
    - Top-level containers – **`frame`** and **`dialog`** are most common (equivalent **`JFrame`** and **`JDialog`**)
    - Secondary-level containers – placed INSIDE top level containers or another secondary container. Most common are **`Panel`** and **`ScrollPane`** (equivalent **`JPanel`**, **`JScrollpane`**)


JAVA GUI APPLICATIONS

Java applications

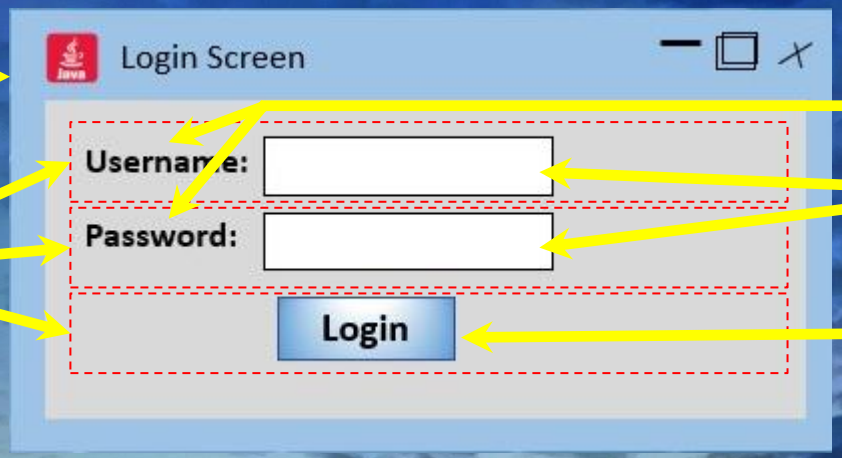
- Extends **`javax.Swing.*`** and **`java.event.*`**
- Makes use of a container class – components are placed inside the container based on a layout.
- With the container, GUI applications are standalone programs that, by default, has a window (with size, border, location etc), titlebar, control section, menu/tool bar (optional) , content area and statusbar
- Typical containers and components are shown below

Containers

JFrame
(Top-level container)

JPanel
(Partitions)

Components



JLabel

JTextField

JButton

JFrames

- Provides main standalone window for a Java GUI application
- Because it is an application, a **`JFrame`** needs the **`main()`** in your code to run (same with **`JDialog`**)
- Has title bar with controls, optional menu bar and content area
- Needs a size!
- Common **`JFrame`** constructors

```
public JFrame()
public JFrame(String title)
public JFrame(GraphicsConfiguration gc)
public JFrame(String,GraphicsConfiguration)
```

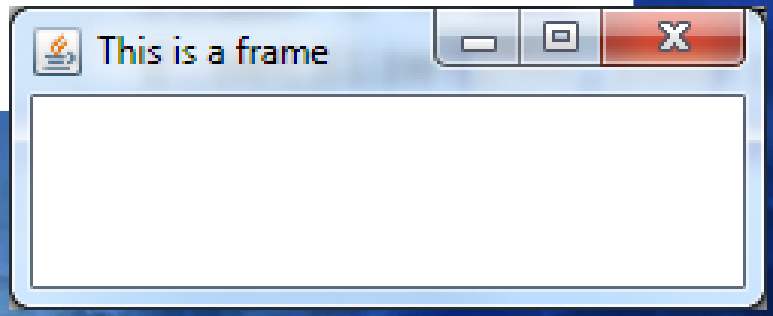
- JFrame objects are ALWAYS invisible during declaration. To show on screen use the **`setVisible`** method.

Ref: https://docs.oracle.com/javase/tutorial/uiswing/components/frame.html

JFrame example 1 (as class instance)

```
import javax.swing.*;

public class TestFrame{
        public static void main(String[] args) {
            JFrame f = new JFrame("This is a frame");
            f.setSize(250,100);
            f.setVisible(true);
            f.setDefaultCloseOperation
                                    (JFrame.EXIT_ON_CLOSE);
        }
}
```



JFrame example 2 (as extends)

```
import javax.swing.*;

public class Test extends JFrame{

        public Test(){
                super("This is a frame");
                setSize(300,300);
                setVisible(true);
                setDefaultCloseOperation
                                (JFrame.EXIT_ON_CLOSE);
        }
        public static void main(String args[]){
                Test m = new Test();
        }
}
```

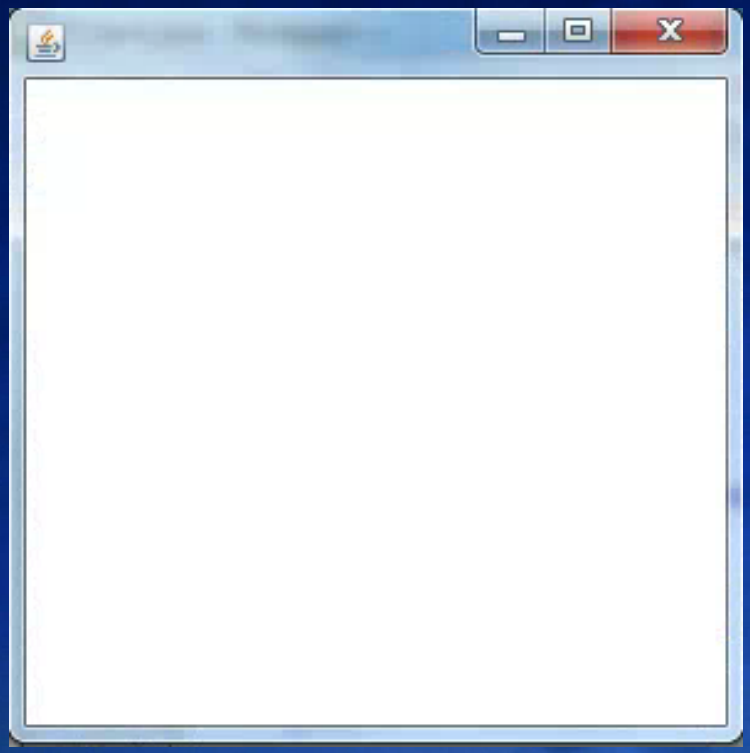
- This time the whole application class inherits and becomes a subclass of Frame

JDialogs

- Provides additional pop-up windows – used as warning or prompt for user to take notice or perform I/O operation
- Has minimal controls for window manipulation (e.g. no maximize)

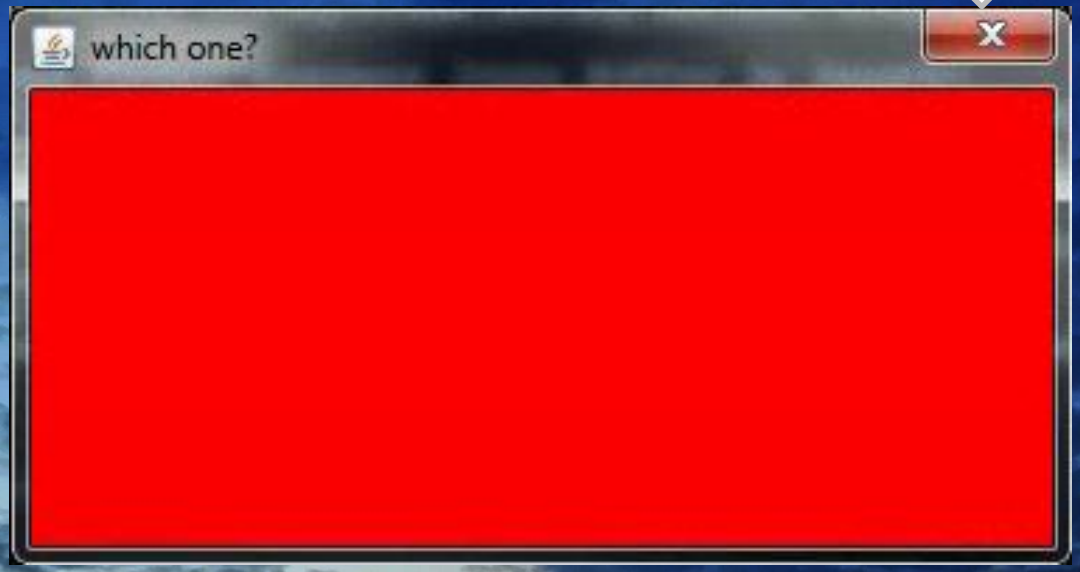
```
import javax.swing.*;
import java.awt.*;
public class Test extends JFrame{
  JDialog d1;
  public Test(){
        d1=new JDialog(this);
        d1.setTitle("which one?");
        d1.setSize(380,200);
        d1.getContentPane().setBackground(Color.red);
        d1.setLocationRelativeTo(null);
        d1.setVisible(true);
        d1.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
        // Note this program does not end correctly!!
        }
  public static void main(String args[]){
        new Test();
  }
}
```

Note how `JFrame` still exists as inheritance superclass. `Dialogs` cannot exist on their own – they usually popup from somewhere – in this case `Frame` is not "displayed".



JFrame has full titlebar control

JDialog only has close



```java
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class SwingJFrameexample extends JFrame{
    public SwingJFrameexample(String title){
        super(title);
        setSize(300,100);

        Container content = getContentPane();
        content.setBackground(Color.white);
        content.setLayout(new FlowLayout());
        content.add(new JButton("Button 1"));
        content.add(new JButton("Button 2"));
        content.add(new JButton("Button 3"));
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public static void main (String args[]){
        JFrame nframe = new SwingJFrameexample("This is JFrame test title");
        nframe.setVisible(true); // to display on screen
    }
}
```

Swing has autoclose

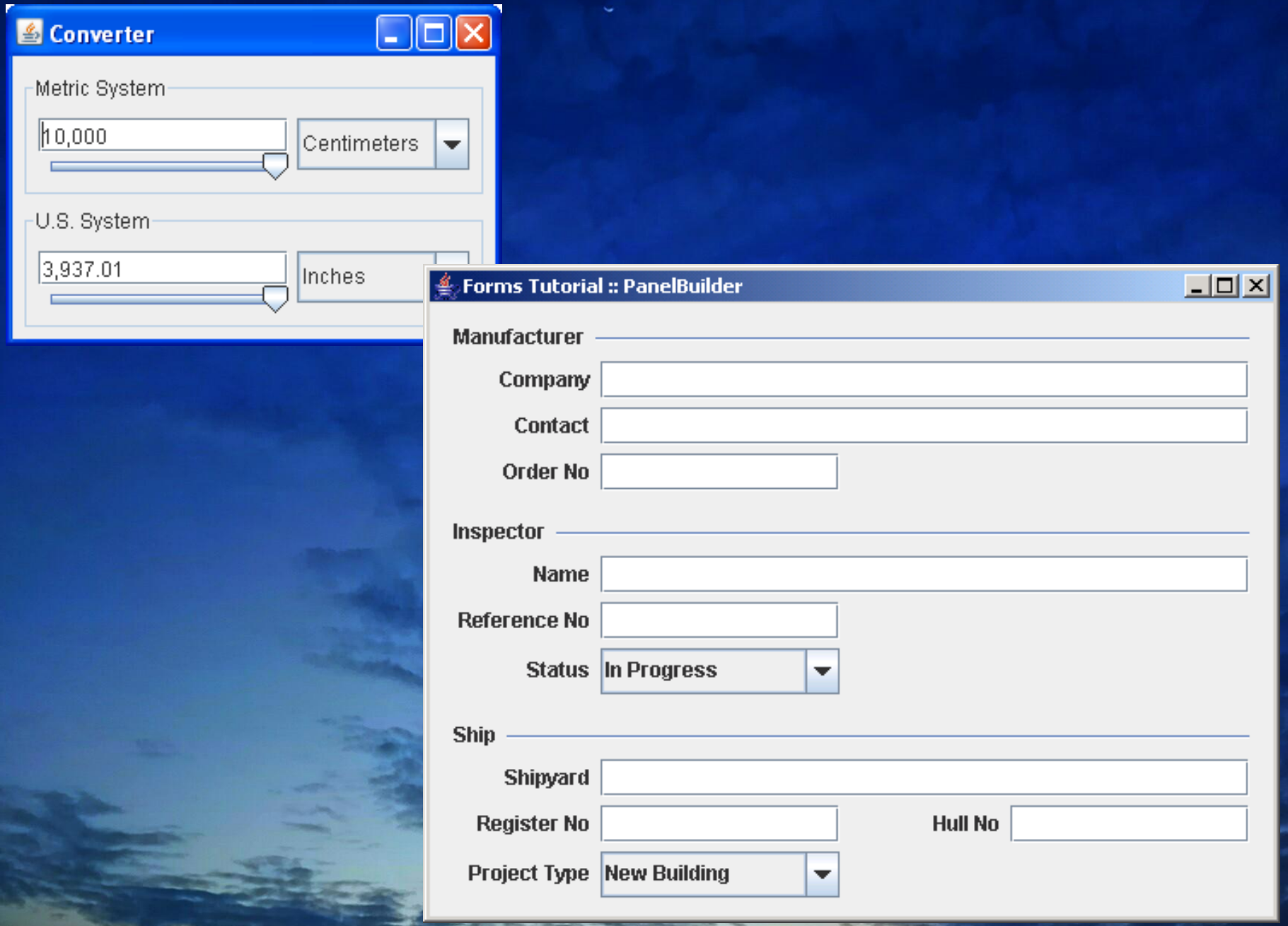
But you still need to END your program!

Secondary-level containers

- Placed INSIDE top level containers or another secondary container - to control layout or to place components in an area using a specific layout style
- Most common are **`JPanel`** and **`JScrollPane`**
- Panel constructors : **`JPanel()`** and **`JPanel(Layout layout)`**
- ScrollPane constructors : **`JScrollPane()`** and **`JScrollPane(int)`**
- Code example

```
JPanel npl = new JPanel();        // create the Panel
npl.add(cbox);                    // Add components into Panel
this.add(npl);                    // Add panel onto this Frame
```

Ref: http://docs.oracle.com/javase/8/docs/api/javax/swing/JPanel.html
http://docs.oracle.com/javase/8/docs/api/javax/swing/JScrollPane.html

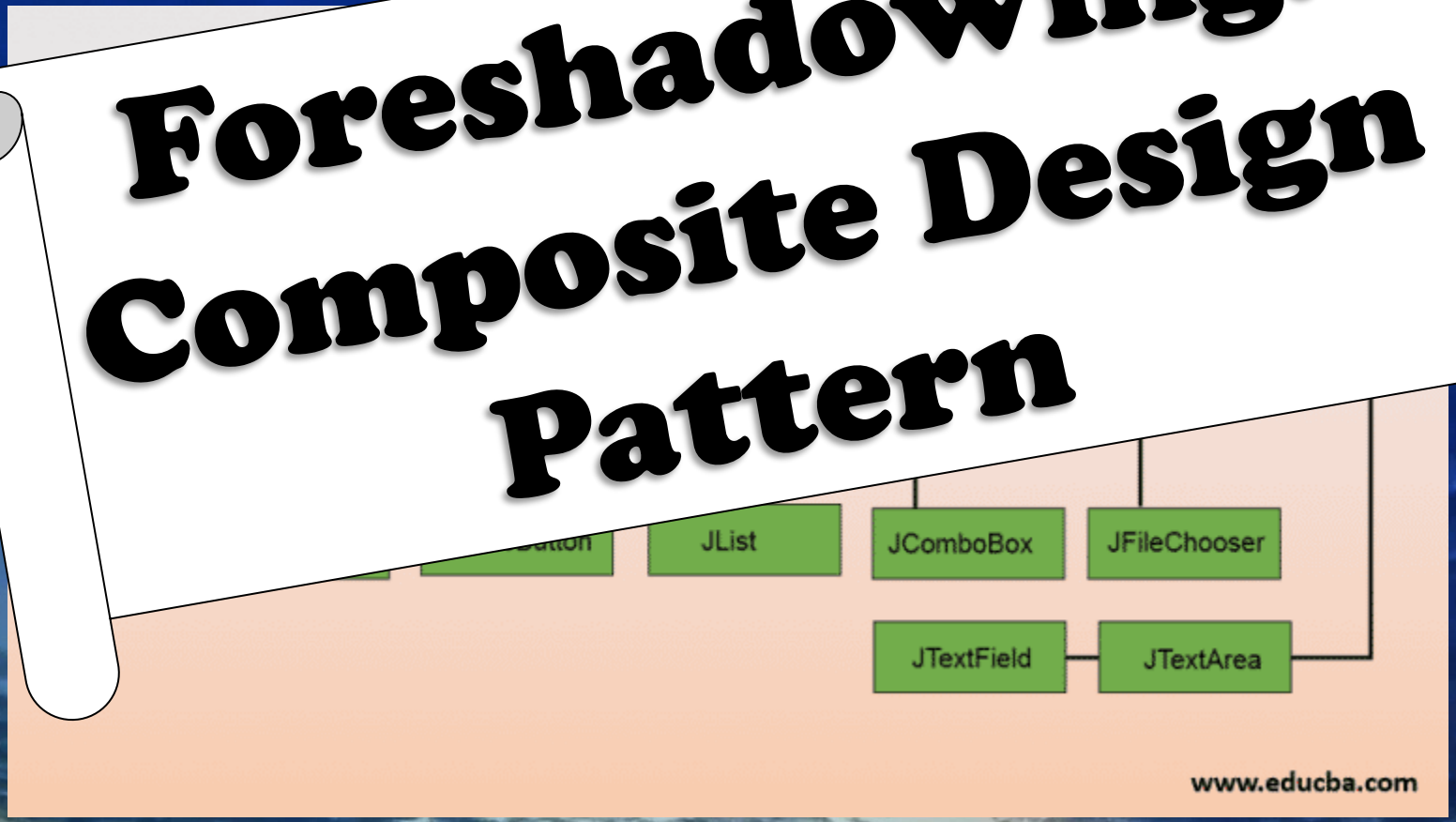

Converter
Metric System
10,000
Centimeters
U.S. System
3,937.01
Inches
Forms Tutorial :: PanelBuilder
Manufacturer
Company
Contact
Order No
Inspector
Name
Reference No
Status
In Progress
Ship
Shipyard
Register No
Hull No
Project Type
New Building

Components

- Classes of ready made and reusable GUI components
- Swing includes **`JButton`**, **`JTextField`**, **`JLabel`**, **`JChec x`** and many other, but these are some of the most common
- All components shown below need

ge

Foreshadowing: Composite Design Pattern



JLabel

- Used to display single line of formatted text
- Six `JLabel` constructors are provided

```
public JLabel(String, int alignment);
public JLabel(String);
public JLabel();
public JLabel(Icon image);
public JLabel(Icon image, int alignment);
public JLabel(String, Icon image, int align);
```

- Syntax example

```
JLabel lbl1=new JLabel ("This is a label");
lbl1.setText("Change the text of the label");
```

Ref: https://docs.oracle.com/javase/8/docs/api/javax/swing/JLabel.html

JButton

- This class creates a labeled button which triggers an action depending on state –normal (non-pushed), highlighted and pushed.
- Requires **`ActionEvent`** to capture event that occurs towards the button – else the button does nothing and is for show only
- Constructors for **`JButton`**

```
public JButton(String btnlbl);
public JButton();
public JButton(Icon icn);
public JButton(String btnlbl, Icon icn);
```

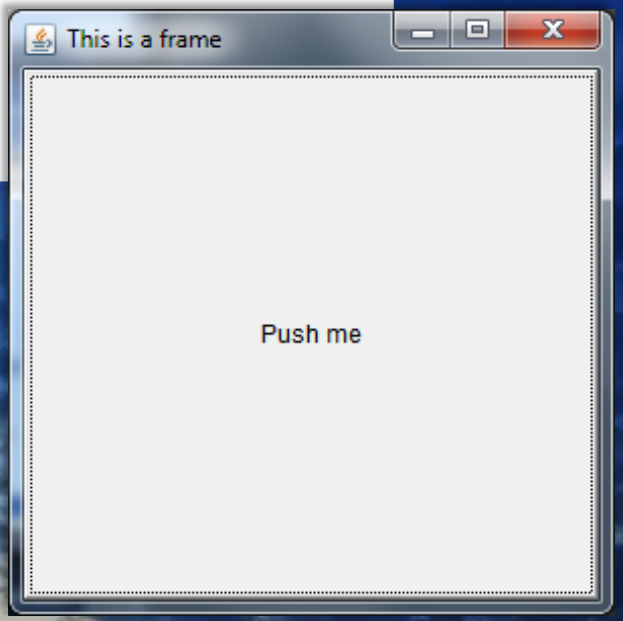
- Syntax example

```
JButton aButton = new JButton("Push me");
```

Ref: https://docs.oracle.com/javase/8/docs/api/javax/swing/JButton.html

```
import javax.swing.*;

public class Test extends JFrame{
	public Test(){
		super("This is a frame");
		JButton aB = new JButton ("Push me");
		this.add(aB);
		setSize(300,300);
		setVisible(true);
		setDefaultCloseOperation
				(JFrame.EXIT_ON_CLOSE);
	}
	public static void main(String args[]){
		Test m = new Test();
	}
}
```



- Note how the button fills the whole frame.
- Without any *layout managers*, components will always try to fill the entire container and overlap each other based on order of declaration

JTextField

- Text fields look like labels but act differently (it can actually replace a label if necessary) – it allows for I/O of text data making it useful for data entry fields
- Constructors

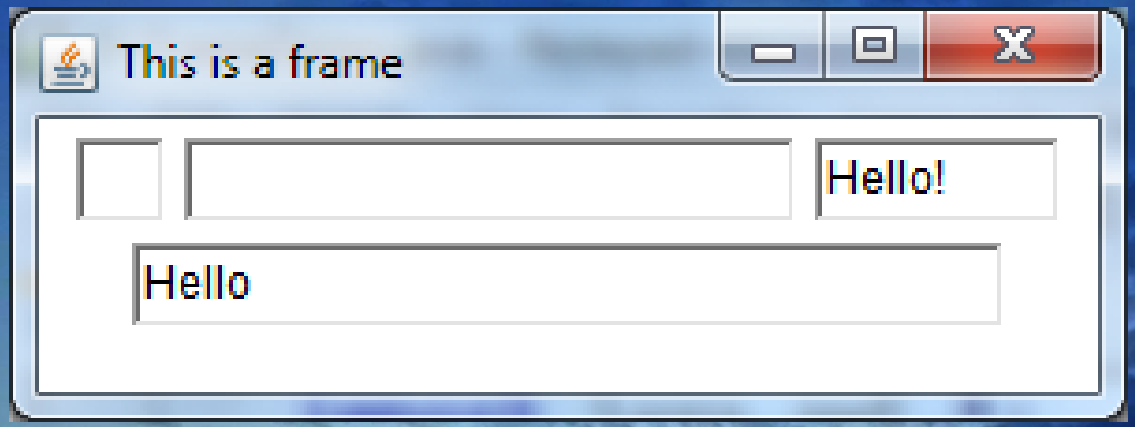
```
public JTextField
   (String strText, int columns);
public JTextField(String strText);
public JTextField(int columns);
public JTextField();
public JTextField
   (Document doc, String strText, int columns);
```

- Syntax example

```
JTextField txtName = new JTextField();
```

Ref: http://docs.oracle.com/javase/8/docs/api/javax/swing/JTextField.html

```
JTextField tf1, tf2, tf3, tf4;
        // a blank text field
        tf1 = new JTextField();
        // blank field of 20 columns
        tf2 = new JTextField("", 20);
        // predefined text displayed
        tf3 = new JTextField("Hello!");
        // predefined text in 30 columns
        tf4 = new JTextField("Hello", 30);
this.add(tf1);
this.add(tf2);
......
```



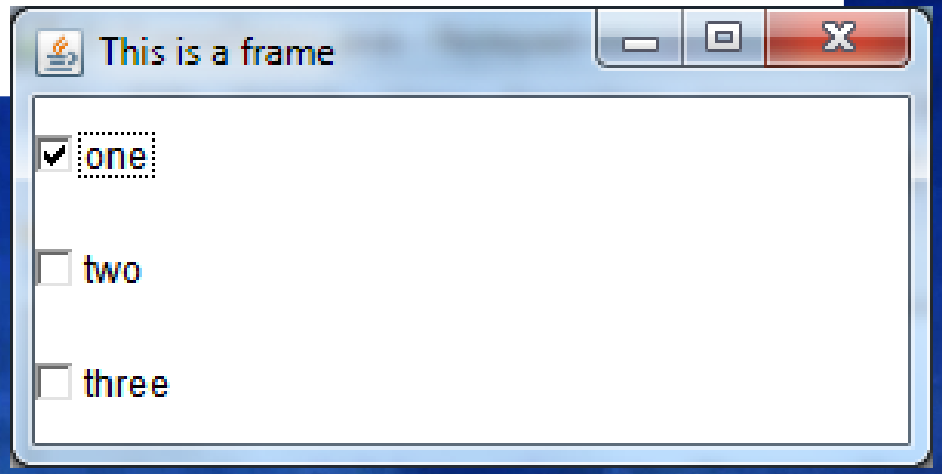
JCheckBox

- Checkboxes are use to provide users with preset options of which the user can choose and select.
- User can select multiple options – good for question fields with multiple answers
- **`JCheckBox`** constructors include

```
public JCheckBox()
public JCheckBox(String label)
public JCheckBox(String, Boolean selected)
public JCheckBox(Icon icon)
public JCheckBox(String text, Icon icon)
```

Ref: https://docs.oracle.com/javase/8/docs/api/javax/swing/JCheckBox.html

```
add(new JCheckBox("one", true));
add(new JCheckBox("two"));
add(new JCheckBox("three"));
setSize(300,150);
setVisible(true);
```

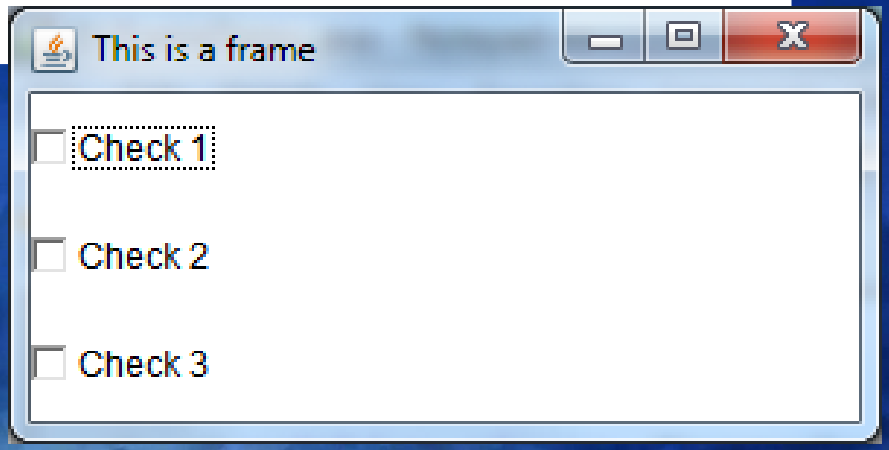


- Note: *Inline* declaration used in this example where no instance identifiers are declared for the checkboxes
- If not comfortable with this declaration, then use normal

```
JCheckBox cbOne = new JCheckBox("one",true)
this.add(cbOne);
```

- Alternative method of declaring multiple checkboxes (or any component)

```
for (int i=0;i<2;i++){
        JCheckBox cbox = new JCheckBox("Check "+i);
        add(cbox);
}
```

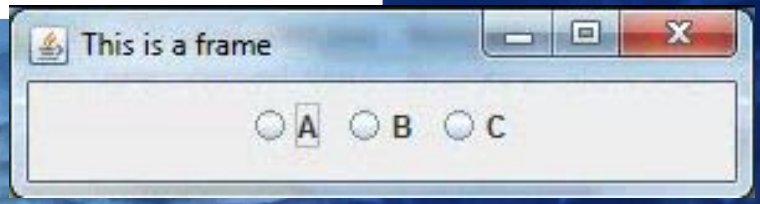


- Note: event handling will be different for components declared in loops because they share the same identifier (in this case all the checkboxes are called **cbox**)

Radio buttons

- **JRadioButton**
- To 'group' multiple radio buttons together, the **ButtonGroup** component is used– you do not need to "add" **ButtonGroup**

```
JRadioButton b1 = new JRadioButton("A");
add(b1);
JRadioButton b2 = new JRadioButton("B");
add(b2);
JRadioButton b3 = new JRadioButton("C");
add(b3);
ButtonGroup bg = new ButtonGroup();
bg.add(b1); bg.add(b2); bg.add(b3);
```



Ref: https://docs.oracle.com/javase/8/docs/api/javax/swing/JRadioButton.html

Choice/Combo Box

- Alternative to radio buttons is to use choice - allows for selection of a single option but saves space compared to radio buttons.
- Choice constructor

```
public Choice()
```

- Syntax example

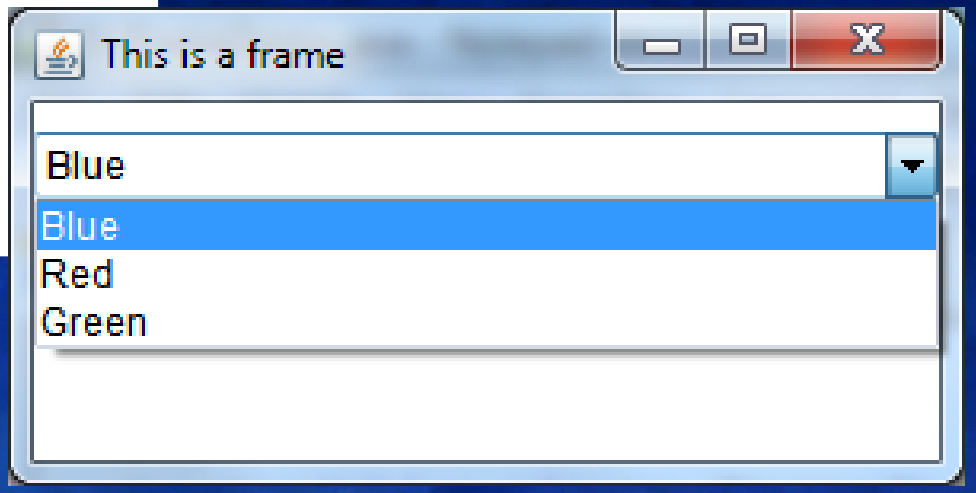
```
Choice colorOption = new Choice();
colorOption.add("Blue");  // accepts String only
```

- Swing replaces choice with **JComboBox**
- Syntax example

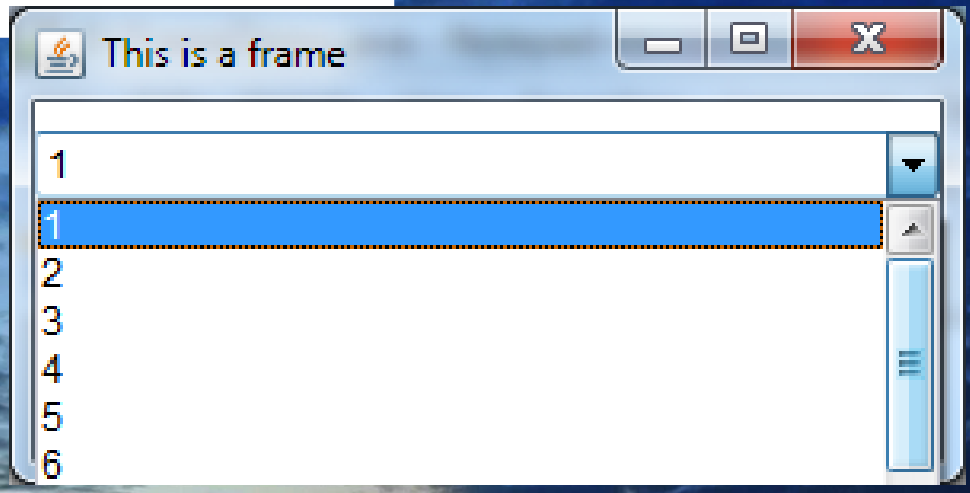
```
String[] petStrings = { "Bird", "Cat", "Dog"};
JComboBox pList = new JComboBox(petStrings);
```

Ref: http://docs.oracle.com/javase/7/docs/api/java/awt/Choice.html
http://docs.oracle.com/javase/7/docs/api/javax/swing/JComboBox.html

```
Choice nc = new Choice();
nc.add("Blue");
nc.add("Red");
nc.add("Green");
add(nc);
```

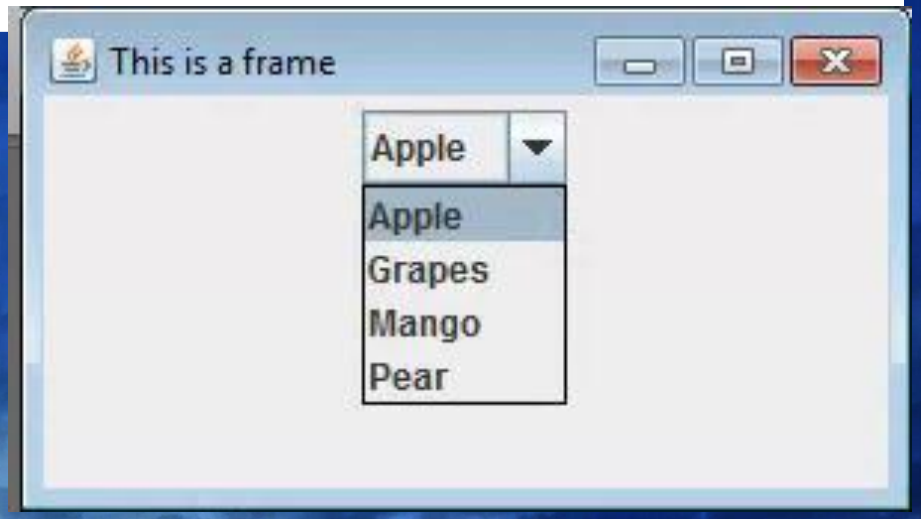



```
Choice nc = new Choice();
for (int i=0;i<10;i++)
        nc.add(Integer.toString(i+1));
add(nc);
```



```
DefaultComboBoxModel fruitsName = new DefaultComboBoxModel();
fruitsName.addElement("Apple");
fruitsName.addElement("Grapes");
fruitsName.addElement("Mango");
fruitsName.addElement("Pear");

JComboBox fruitCombo = new JComboBox(fruitsName);
fruitCombo.setSelectedIndex(0);
```



JTextArea

- JTextArea is fairly similar to the JTextField but can span multiple lines
- JTextArea has additional options to set size and scroll bar options
- JTextArea common constructors

```
public JTextArea()
public JTextArea(int,int)
public JTextArea(String)
public JTextArea(String, int row, int col)
public JTextArea(Document)
```

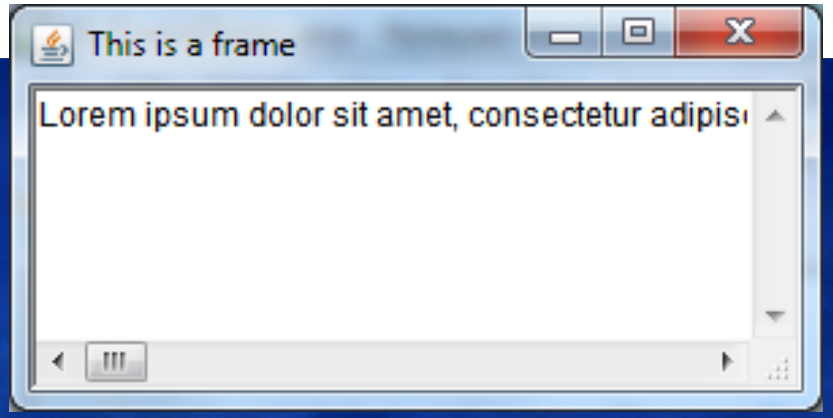
- Syntax example

```
JTextArea nt = new JTextArea("Hello",5,40);
```

Ref: http://docs.oracle.com/javase/8/docs/api/javax/swing/JTextArea.html

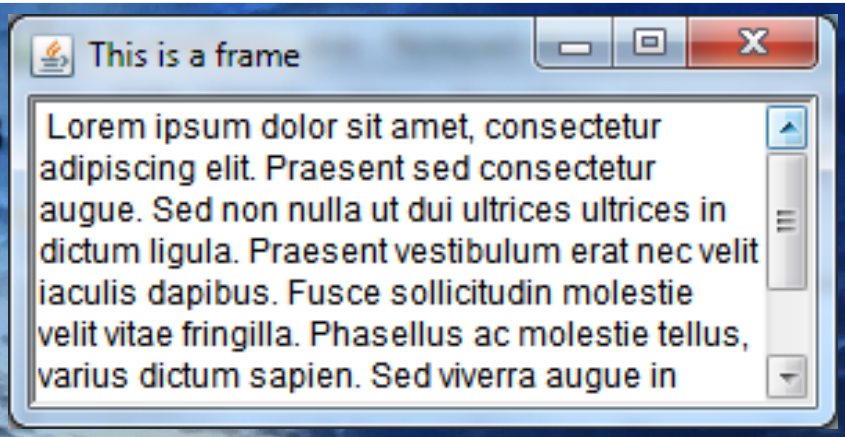
```
String sample = new String("Lorem ipsum dolor sit amet.....");
JTextArea ta = new JTextArea(sample, 4,30);
add(ta);
```

By default, a text area does not auto "wrap" text so any text string longer than the area creates scroll bars




```
String sample = new String("Lorem ipsum dolor sit amet.....");
JTextArea ta = new JTextArea(sample,
                    4,30,TextArea.SCROLLBARS_VERTICAL_ONLY);
add(ta);
```

Note: In this example, no layout managers are used so the components fill the container and ignore the size parameters



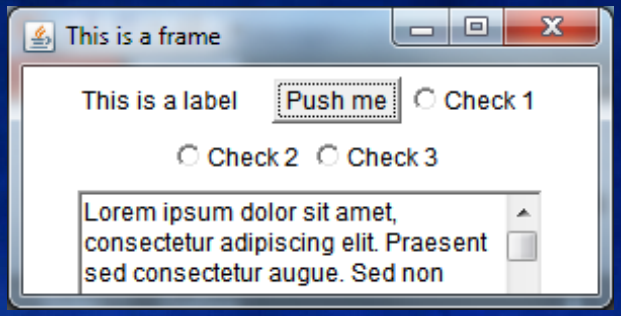
Other GUI components

- Other useful components but not covered here
  - JScrollbar → adds scrollbar(s) to a container
  - JFileChooser → built-in file open/save dialog (Swing only)
  - JColorChooser → color palette (Swing only)
  - MDI → multiple document interface (i.e. window in window)
  - JOptionPane → alternative to JDialog
  - JEditorPane → text editor with support for HTML elements

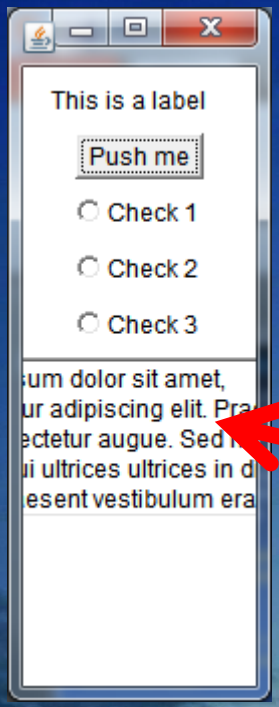
Layout Managers

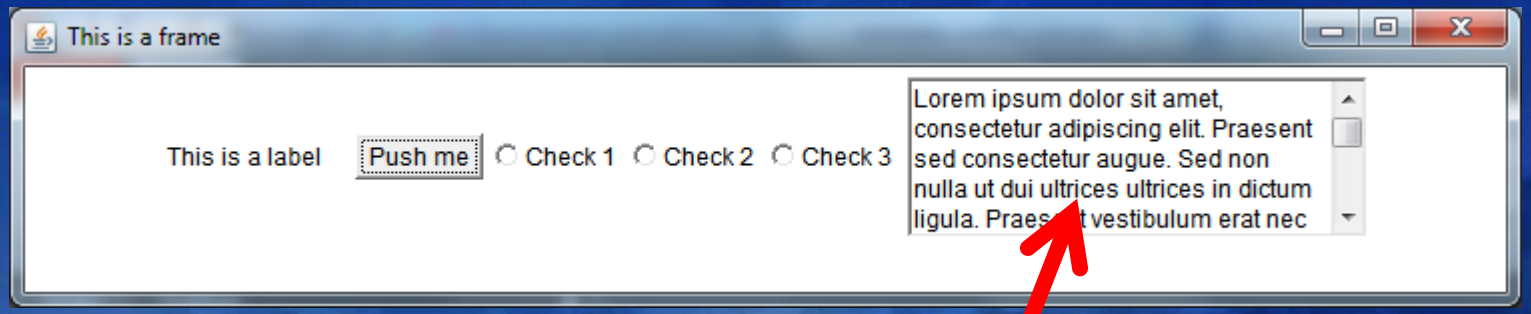
- Swing has many layout managers; 3 are most commonly used.
  - **`FlowLayout`**, **`GridLayout`**, **`BorderLayout`**, **`GridBagLayout`**, **`BoxLayout`**, **`CardLayout`**, **`GroupLayout`**, **`SpringLayout`**
- Each container usually has its default layout if you do not specify (content pane is border, panel is flow, etc. → see javadocs)
- **`FlowLayout`** – arranges components in order they are declared left-to-right top-to-bottom
- **`GridLayout`** – arranges components in order they are declared based on available grid cells (increased when necessary)
- **`BorderLayout`** – arranges components based on relativity to compass direction (NORTH, SOUTH, WEST. EAST, CENTER)
- The following examples show how Flow, Grid and Border layout managers will handle different components on a Frame.

```
setLayout(new FlowLayout());
```



If application is resized, **`FlowLayout`** manager re-flows items accordingly

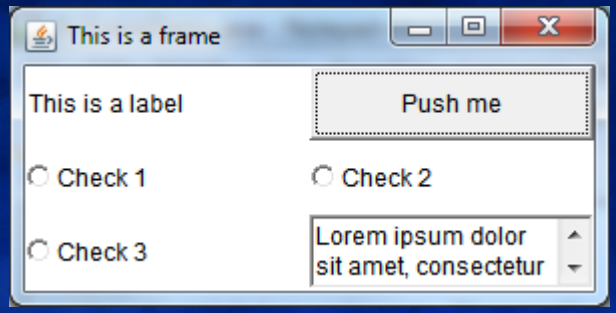




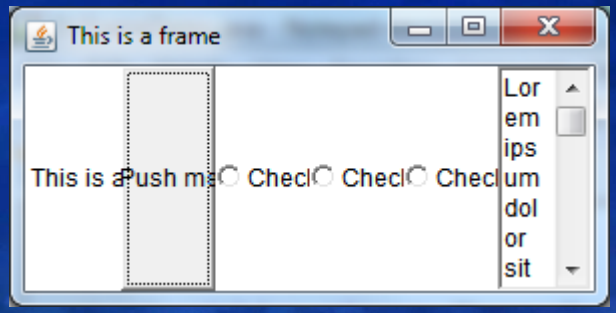
Notice how **`JTextArea`** now follows the size declaration no matter the application size

```
setLayout(new GridLayout(3,1));
```

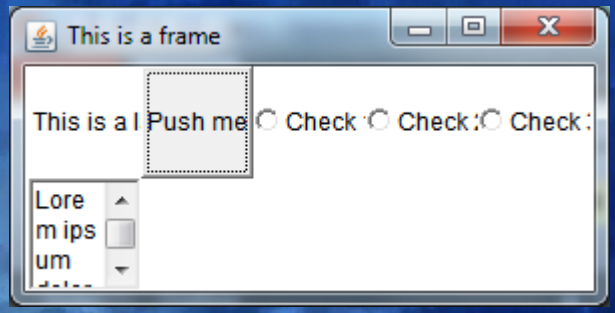
(rows,columns)




```
setLayout(new GridLayout(1,3));
```




```
setLayout(new GridLayout(0,5));
```

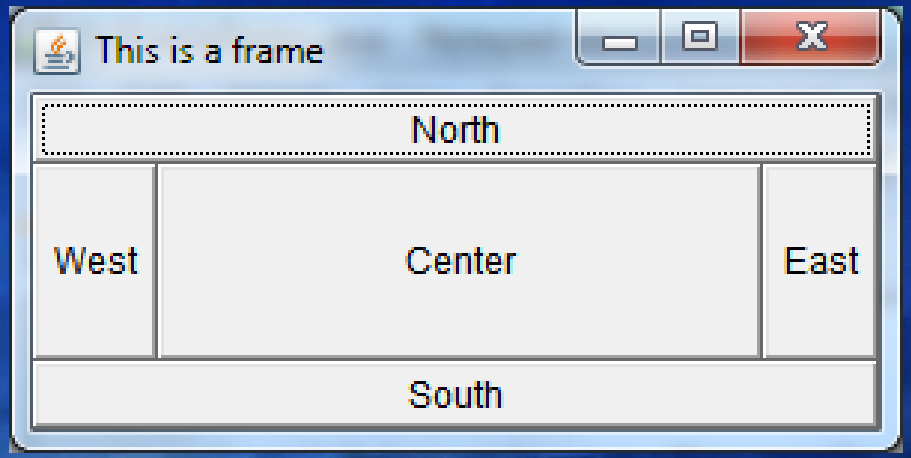


When both the number of rows and the number of columns have been set to non-zero values, either by a constructor or by the setRows and setColumns methods, the number of columns specified is ignored.

Instead, the number of columns is determined from the specified number of rows and the total number of components in the layout.

```
setLayout(new BorderLayout());
add(new Button("North"), BorderLayout.NORTH);
add(new Button("South"), BorderLayout.SOUTH);
add(new Button("East"), BorderLayout.EAST);
add(new Button("West"), BorderLayout.WEST);
add(new Button("Center"), BorderLayout.CENTER);
```

If the window is enlarged, the center area gets as much of the available space as possible.



- Note that this method is deprecated but still supported.
- Latest versions of Java recommend PAGE_START, LINE_START, CENTER, LINE_END and PAGE_END to replace North, West, Center, East and South.


EVENT HANDLING

Event Handling

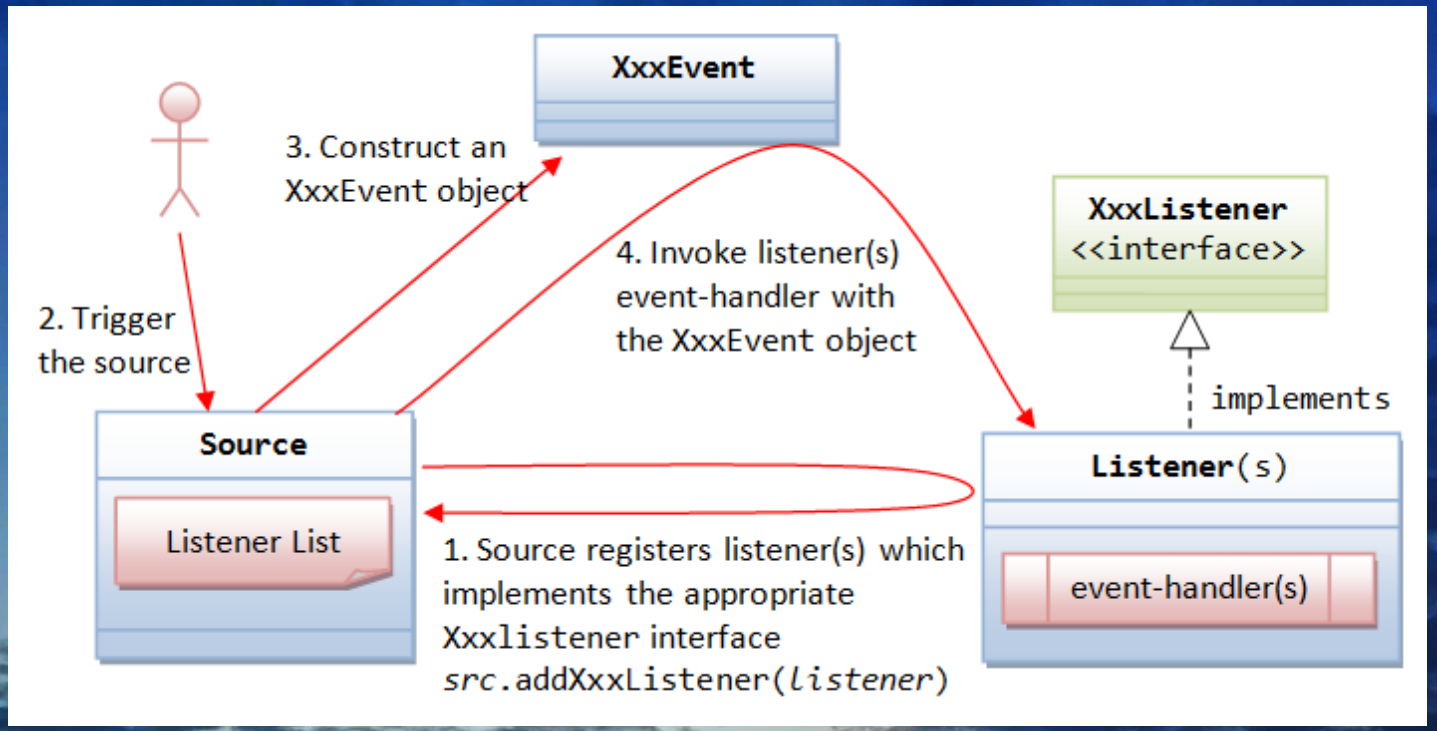
- Everything that occurs in a GUI is an event
- To handle user interactions, you need to add in triggers (event source), targets (event listener) and types (event object)
- Two general groups in Swing: component events and I/O events
  - component events handle interactions with GUI components (e.g. button is clicked, text area is filled, checkbox is unchecked etc)
  - I/O events handle input and output events from users through mouse and keyboard (or touch) --> "`implements MouseListener,KeyListener`"
- Event handling requires you to overwrite ALL functions from the implemented class

I/O events

- Represents events that occur from mouse and keyboard trigger (touch is not included in AWT)
- Event handling is provided by interface classes – divided into MouseEvents. MouseListener and KeyEvents
- Because they are interface classes, they must be `implements` into program code
- Mouse events have two interface classes because one tracks for movement whilst the other tracks for button presses
- Java Listeners are examples of the Observer Design Pattern that you will learn later in the lectures on Design Patterns.

Handling events

- `Implements` the listener you want to use in your program class
- Create and add your component you want to track
- Add the listener to that component
- Add the event handler which triggers when the listener 'hears' something and sends an event message
- Complete the event handler code to handle the event that was triggered.



Mouse events

- MouseListener

```
public void mouseClicked(MouseEvent evt)
public void mouseExited(MouseEvent evt)
public void mouseReleased(MouseEvent evt)
public void mousePressed(MouseEvent evt)
public void mouseEntered(MouseEvent evt)
```

- MouseMotionListener

```
public void mouseDragged(MouseEvent e)
public void mouseMoved(MouseEvent e)
```

Example Mouse event handling

- Basic code with no event handling

```
import javax.swing.*;
import java.awt.*;

public class FrameDemo extends JFrame{

        public FrameDemo(){
        super("This is a frame");
                setSize(300,150);
                setVisible(true);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
}
```

- Add in listener we want (this example tracks mouse movement) AND event handling package

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class FrameDemo extends JFrame implements MouseListener{
	public FrameDemo(){
	super("This is a frame");
		setSize(300,150);
		setVisible(true);
	}
	public static void main(String args[]){
		FrameDemo m = new FrameDemo();
	}
}
```

- Override all abstract methods from interface class (add them all in!)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class FrameDemo extends JFrame implements MouseListener
{       public FrameDemo(){
        super("This is a frame");
                setSize(300,150);
                setVisible(true);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void mouseClicked(MouseEvent evt){};
        public void mouseExited(MouseEvent evt){};
        public void mouseReleased(MouseEvent evt){};
        public void mousePressed(MouseEvent evt){};
        public void mouseEntered(MouseEvent evt){};

}
```

- Override the abstract methods we want to use

```
public class FrameDemo extends JFrame implements MouseListener{
        public FrameDemo(){
        super("This is a frame");
                setSize(300,150);
                setVisible(true);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void mouseClicked(MouseEvent evt){};
        public void mouseExited(MouseEvent evt){
                getContentPane().setBackground(Color.white);};
        public void mouseReleased(MouseEvent evt){};
        public void mousePressed(MouseEvent evt){};
        public void mouseEntered(MouseEvent evt){
                getContentPane().setBackground(Color.red);};

}
```

- Add the trigger to the component

```
public class FrameDemo extends JFrame implements MouseListener{
        public FrameDemo(){
        super("This is a frame");
                setSize(300,150);
                setVisible(true);
                addMouseListener(this);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void mouseClicked(MouseEvent evt){};
        public void mouseExited(MouseEvent evt){
                getContentPane().setBackground(Color.white);};
        public void mouseReleased(MouseEvent evt){};
        public void mousePressed(MouseEvent evt){};
        public void mouseEntered(MouseEvent evt){
                getContentPane().setBackground(Color.red);};
}
```

Example Mouse event handling 2

```
public class FrameDemo extends JFrame implements MouseListener{
        Label nl = new Label("This is a label");
        public FrameDemo(){
                setLayout(new FlowLayout());
                add(nl);
                setSize(300,150);
                setVisible(true);
                addMouseListener(this);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void mouseClicked(MouseEvent evt){
                int x = evt.getX();
                int y = evt.getY();
                nl.setText("Clicked: " + x + "," +y);
                };
        public void mouseExited(MouseEvent evt){};
        public void mouseReleased(MouseEvent evt){};
        public void mousePressed(MouseEvent evt){};
        public void mouseEntered(MouseEvent evt){};
}
```

Keyboard events

- Keyboards cannot move so they only have one interface class for events
- Keyboard events are triggered whenever the keyboard is used (does not have to be within a component like a text field)
- Useful for handling events in applications with no discrete components for keyboard to interact with (e.g. games)
- KeyEvents

```
public void keyPressed(KeyEvent e){};
public void keyReleased(KeyEvent e){};
public void keyTyped(KeyEvent e){};
```

Keyboard event example

```
public class FrameDemo extends JFrame implements KeyListener{
        Label nl = new Label("This is a label");
        public FrameDemo(){
                super("This is a frame");
                setLayout(new FlowLayout());
                add(nl);
                setSize(300,150);
                setVisible(true);
                addKeyListener(this);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void keyPressed(KeyEvent evt){
                int keyPressed=evt.getKeyCode();
                nl.setText("Key = "+(char)keyPressed);
        }
        public void keyReleased(KeyEvent evt){};
        public void keyTyped(KeyEvent evt){};
}
```

Component events

- Makes use of a set of Listeners → ActionListener, ItemListener, and AdjustmentListener - each for a specific task
  - ActionListener 'listens' for actions like button presses and textfield changes
  - ItemListener 'listens' for actions like selections and check/uncheck operations on multiple items (lists, choices, checkboxes)
  - AdjustmentListener 'listens' for actions on scrollbars

Component event listeners

- How to use?
  - implement the interface class ActionListener, ItemListener and/or AdjustmentListener to your class
  - Attach a source trigger to a component (i.e. addActionListener, addItemListener, addAdjustmentListener)
  - override ALL methods from interface class
  - capture event (ActionEvent, ItemEvent, AdjusmentEvent)
  - perform operation against event captured
- Some events can be triggered by the component themselves (e.g. similar to the mouseEntered, mouseExited previously) and can also be captured
  - Example: Button also has mouseEntered and MouseExited triggers which can be used to highlight mouse overs

ActionListener

- Works on buttons, fields etc.
- To use, implements ActionListener in the program code
- Add the ActionListener to the object to which an event will be triggered
- Overload all abstract methods from ActionListener in program code (i.e actionPerformed)
- Handle the triggered event in the chosen method

ActionListener example

```
public class FrameDemo extends JFrame implements ActionListener{
        private Button btnCount;

        public FrameDemo(){
                setLayout (new FlowLayout());
                btnCount = new Button("Close");
                add(btnCount);
                setSize(300,150);
                setVisible(true);
                btnCount.addActionListener(this);
        }
        public static void main(String args[]){
                FrameDemo m = new FrameDemo();
        }
        public void actionPerformed(ActionEvent evt){
                System.exit(0);
        }
}
```

ActionListener example 2

- What if multiple buttons declared under one identifier?

```
for (int i=0;i<3;i++){
        Button btnarray = new Button("Button "+i);
        add(btnarray);
        btnarray.addActionListener(this);
}
```

- Use method to extract button details

```
public void actionPerformed(ActionEvent evt){
        String btn=evt.getActionCommand();
        if (btn.equals("Button 0"))
                System.exit(0);
        else if (btn.equals("Button 1"
                lbl.setText("Button 1
        else if (btn.equals("Button 2
                lbl.setText("Button 2 pressed");
        else if (btn=="Close")
                System.exit(0);
        }
```

Bad design due to if-else.
Later: polymorphism!

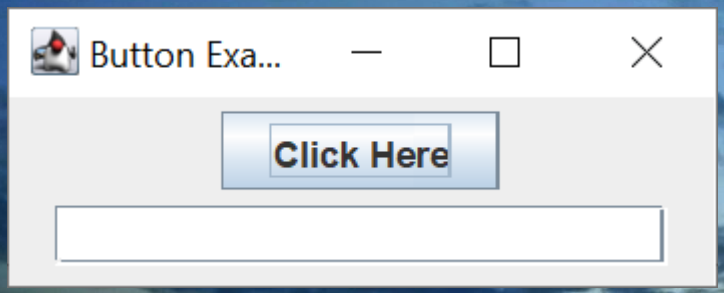
Button Example

```
import java.awt.event.*;
import java.awt.*;
import javax.swing.*;
public class ButtonExample {
    public static void main(String[] args) {
        JFrame f=new JFrame("Button Example");
        final JTextField tf=new JTextField("", 20);
        tf.setBounds(50,50, 150,20);
        JButton b=new JButton("Click Here");
        b.setBounds(50,100,95,30);
        b.addActionListener(new MyButtonListener(tf));
        f.add(b);f.add(tf);
        f.setSize(250,100);
        f.setLayout(new FlowLayout());
        f.setVisible(true);
    }
}
```

```
class MyButtonListener implements ActionListener
{   JTextField tf;

    public MyButtonListener(JTextField tf)
    {   this.tf = tf;
    }

    public void actionPerformed(ActionEvent e)
    {  tf.setText("Welcome to OOAD.");
    }
}
```



Click

Item and Adjustment listeners

- Similar method of implementing listeners – just different interface classes and methods to override and on different components
- Not all components will work correctly with listeners (e.g. you should not addActionListener to a JFrame)

Ref:
http://docs.oracle.com/javase/8/docs/api/java/awt/event/AdjustmentListener.html

JavaFX

- Newer way to do Java GUIs
- public class HelloWorld extends Application

JavaFX

Instead of main()

```
@Override
public void start(Stage primaryStage)
{   primaryStage.setTitle("Hello World!");
    Button btn = new Button();
    btn.setText("Say 'Hello World'");
    btn.setOnAction(new ButtonPressed());

    StackPane root = new StackPane();
    root.getChildren().add(btn);
    primaryStage.setScene(new Scene(root, 300, 250));
    primaryStage.show();
}
```

JavaFX

Observer Pattern

```
private class ButtonPressed implements EventHandler<ActionEvent>
{       @Override
        public void handle(ActionEvent event) {
                System.out.println("Hello World!");
        }
}
```

JavaFX

- Tutorial available at https://docs.oracle.com/javafx/2/get_started/jfxpub-get_started.htm

We're sticking with Swing… why?

- This is an OOAD course, not a Java course.
- Stable and Simple for Basic GUI Requirements
  - Swing's Stability
  - Focus on OO Concepts
- Minimal Setup Complexity
  - Swing is bundled with the JDK
  - Avoiding JavaFX Configuration Overhead
- Sufficient for Teaching Design Patterns
  - Design patterns, MVC, etc.
  - Event-driven programming.
- JavaFX learning curve.
  - Complexity
  - Unnecessary Detail for OOAD Focus

Android

- There is no **`main()`**
- **public class** MainActivity **extends** AppCompatActivity

```
package com.example.myfirstapp;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

GUI is specified by XML

```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context="com.example.myfirstapp.MainActivity">

   <EditText
      android:id="@+id/editText"
      android:layout_width="0dp"
      android:layout_height="wrap_content"
      android:layout_marginTop="16dp"
      android:ems="10"
      android:hint="@string/edit_message"
      android:inputType="textPersonName"
      app:layout_constraintLeft_toLeftOf="parent"
      app:layout_constraintTop_toTopOf="parent"
      app:layout_constraintRight_toLeftOf="@+id/button"
      android:layout_marginLeft="16dp" />

   <Button
      android:id="@+id/button"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="@string/button_send"
      app:layout_constraintBaseline_toBaselineOf="@+id/editText"
      app:layout_constraintLeft_toRightOf="@+id/editText"
      app:layout_constraintRight_toRightOf="parent"
      android:layout_marginLeft="16dp"
      android:layout_marginRight="16dp" />
</android.support.constraint.ConstraintLayout>
```

Strings

- Kept in strings.xml for easy translation without needing to access the source code.

```
<resources>
    <string name="app_name">My First App</string>
    <string name="edit_message">Enter a message</string>
    <string name="button_send">Send</string>
</resources>
```

There's more!

- What you need to do on your own
  - Try out all the code examples shown today on your own
  - Refer to the online documentation at https://docs.oracle.com/javase/8/docs/api/index.html?javax/swing/package-summary.html
  - Read up on the listeners
  - Swing tutorial at https://docs.oracle.com/javase/tutorial/uiswing/
  - JavaFX tutorial at https://docs.oracle.com/javafx/2/get_started/jfxpub-get_started.htm

There's more!

NOTE: The content in this set of slides is NOT ENOUGH to cover the entire content of Java GUI/event programming .

You MUST put in your own effort to research other methods not covered here that you will need.